```
#!/usr/bin/env python3
"""
Kalahari Magnetic and Soil Survey Node - data processing

Usage:
    python map_survey.py survey.csv            # process a real survey file
    python map_survey.py --demo                # synthetic data with a buried dipole

Outputs (in the current folder):
    survey_maps.png     magnetic anomaly map and soil conductivity map
    anomalies.csv       stations flagged as anomalous
"""
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

Z_THRESHOLD = 2.0   # flag stations more than 2 standard deviations from the trend
```

```
def make_demo(path="demo_survey.csv",
seed=7):
    """Synthetic grid survey over Gaborone
with one buried magnetic body.
    Illustrative only: not real field
data."""
    rng = np.random.default_rng(seed)
    lat0, lon0 = -24.6282, 25.9231
    xs, ys = np.meshgrid(np.arange(0, 200,
10), np.arange(0, 200, 10))
    x, y = xs.ravel(), ys.ravel()
    lat = lat0 + y / 111_320.0
    lon = lon0 + x / (111_320.0 *
np.cos(np.radians(lat0)))
    r2 = (x - 120) ** 2 + (y - 90) ** 2
    h = 25.0
    dipole = 900.0 * (2 * h**2 - r2) / (r2
+ h**2) ** 2.5 * h**3     # nT,
illustrative
    regional = 30000 + 0.8 * x - 0.5 *
y                               # gentle
gradient
    btotal = regional + dipole +
rng.normal(0, 25, x.size)
    conductive = 1.0 / (1 + np.exp(-(120 -
x) / 15)) * 0.0 + np.exp(-r2 / (2 * 30**2))
    rs = 40000 / (1 + 3.5 * conductive) *
rng.normal(1, 0.06, x.size)
    temp = rng.normal(27, 0.6, x.size)
    ec = 1e6 / rs
    ec25 = ec / (1 + 0.02 * (temp - 25))
    df = pd.DataFrame({
        "station": np.arange(1, x.size +
1), "utc": "demo", "lat": lat, "lon": lon,
```

```
        "hdop": 1.2, "sats": 9, "bx_nT":
btotal, "by_nT": 0, "bz_nT": 0,
        "btotal_nT": btotal, "rs_ohm": rs,
"ec_uS": ec, "temp_C": temp,
        "ec25_uS": ec25, "fix_ok": 1,
    })
    df.to_csv(path, index=False)
    return path


def to_local_metres(df):
    lat0, lon0 = df["lat"].mean(),
df["lon"].mean()
    df["x_m"] = (df["lon"] - lon0) *
111_320.0 * np.cos(np.radians(lat0))
    df["y_m"] = (df["lat"] - lat0) *
111_320.0
    return df


def remove_trend(x, y, v):
    """Fit and subtract a plane (regional
field) from v."""
    a = np.column_stack([x, y,
np.ones_like(x)])
    coef, *_ = np.linalg.lstsq(a, v,
rcond=None)
    return v - a @ coef


def main():
    if len(sys.argv) < 2:
        print(__doc__)
        sys.exit(1)
```

```
    path = make_demo() if sys.argv[1] == "-
-demo" else sys.argv[1]

    df = pd.read_csv(path)
    df = df[df["fix_ok"] ==
1].dropna(subset=["btotal_nT"]).copy()
    if len(df) < 10:
        sys.exit("Need at least 10 stations
with a GPS fix.")
    df = to_local_metres(df)

    df["b_anom_nT"] =
remove_trend(df["x_m"].values,
df["y_m"].values, df["btotal_nT"].values)
    df["b_z"] = (df["b_anom_nT"] -
df["b_anom_nT"].mean()) /
df["b_anom_nT"].std()
    ec = df["ec25_uS"].fillna(df["ec_uS"])
    df["ec_z"] = (ec - ec.mean()) /
ec.std()
    df["flag"] = (df["b_z"].abs() >
Z_THRESHOLD) | (df["ec_z"] > Z_THRESHOLD)

    df[df["flag"]].to_csv("anomalies.csv",
index=False)
    print(f"{len(df)} stations processed,
{int(df['flag'].sum())} flagged ->
anomalies.csv")

    fig, axes = plt.subplots(1, 2, figsize=
(11, 4.6), constrained_layout=True)
    lim =
np.nanmax(np.abs(df["b_anom_nT"]))
    s1 = axes[0].scatter(df["x_m"],
```

```
df["y_m"], c=df["b_anom_nT"],
cmap="RdBu_r",
                          vmin=-lim,
vmax=lim, s=90, edgecolor="k",
linewidth=0.3)
    axes[0].set_title("Magnetic anomaly
(trend removed)")
    fig.colorbar(s1, ax=axes[0],
label="nT")
    s2 = axes[1].scatter(df["x_m"],
df["y_m"], c=ec, cmap="viridis", s=90,
                          edgecolor="k",
linewidth=0.3)
    axes[1].set_title("Soil conductance
index (temperature corrected)")
    fig.colorbar(s2, ax=axes[1],
label="uS")
    for ax in axes:
        flagged = df[df["flag"]]
        ax.scatter(flagged["x_m"],
flagged["y_m"], s=170, facecolors="none",
                   edgecolors="black",
linewidths=1.4)
        ax.set_xlabel("Easting (m)")
        ax.set_ylabel("Northing (m)")
        ax.set_aspect("equal")
    fig.suptitle("Kalahari survey node -
station map (ringed = flagged)")
    fig.savefig("survey_maps.png", dpi=160)
    print("Saved survey_maps.png")
```

```
if __name__ == "__main__":
    main()
```